

E-Commerce Sales Analysis Database

Assignment Overview

This assignment is based on an e-commerce business scenario where data analysis is performed using SQL. The database is designed to simulate real-world online shopping operations including customers, products, orders, and order items.

The goal of this assignment is to practice SQL concepts such as:

- Data retrieval using SELECT
- Filtering using WHERE
- Aggregation using GROUP BY
- Joins and table relationships
- Constraints and keys
- CASE statements
- Transactions and ACID properties
- Query optimization using indexes

This assignment helps in understanding how SQL is used in business environments to generate insights and support decision-making.

1. Business Scenario

You are working as a **Junior Data Analyst at ShopEase**, a mid-sized e-commerce company that sells:

- Electronics
- Clothing
- Home products

The management team wants to analyze:

- Sales performance
- Customer buying behavior
- Product demand
- Revenue trends

- Order statuses

To perform this analysis, a relational database has been created.

2. Database Name

shopease_db

This database stores all business-related transactional and customer data.

3. Database Structure

The database consists of four main tables:

Table Name	Description
customers	Stores customer details
products	Stores product details
orders	Stores order information
order_items	Stores product-level order details

4. Database Creation Query

```
CREATE DATABASE shopease_db;
```

```
USE shopease_db;
```

Explanation

- CREATE DATABASE creates a new database.
 - USE selects the database for executing queries.
-

5. Customers Table

Query

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,
```

```
email VARCHAR(100) UNIQUE NOT NULL,  
city VARCHAR(50) NOT NULL,  
state VARCHAR(50) NOT NULL,  
join_date DATE NOT NULL,  
is_premium BOOLEAN DEFAULT FALSE  
);
```

Explanation

This table stores customer information.

Constraints used:

- PRIMARY KEY → uniquely identifies customer
- NOT NULL → prevents empty values
- UNIQUE → prevents duplicate emails
- DEFAULT → assigns false if no premium status given

Customers Sample Data

```
SELECT * FROM customers;
```

customer_id	first_name	last_name	city	state	join_date	is_premium
101	Aarav	Sharma	Mumbai	Maharashtra	2024-01-15	TRUE
102	Priya	Patel	Ahmedabad	Gujarat	2024-02-20	FALSE
103	Rohan	Gupta	Delhi	Delhi	2024-03-10	TRUE
104	Sneha	Reddy	Hyderabad	Telangana	2024-04-05	FALSE
105	Vikram	Singh	Jaipur	Rajasthan	2024-05-12	TRUE
106	Ananya	Iyer	Chennai	Tamil Nadu	2024-06-18	FALSE
107	Karan	Mehta	Pune	Maharashtra	2024-07-22	TRUE
108	Divya	Nair	Kochi	Kerala	2024-08-30	FALSE

6. Products Table

Query

```
CREATE TABLE products (  
  product_id INT PRIMARY KEY,  
  product_name VARCHAR(100) NOT NULL,  
  category VARCHAR(50) NOT NULL,  
  brand VARCHAR(50) NOT NULL,  
  unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),  
  stock_qty INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0)  
);
```

Explanation

Stores all product details.

Constraints:

- Price must be greater than 0
- Stock cannot be negative

Products Sample Data

```
SELECT * FROM products;
```

product_id	product_name	category	brand	unit_price	stock_qty
201	Wireless Earbuds	Electronics	BoAt	1499	250
202	Cotton T-Shirt	Clothing	Levis	799	500
203	Smart Watch	Electronics	Noise	2999	150
204	Running Shoes	Clothing	Nike	4599	120
205	Bluetooth Speaker	Electronics	JBL	3499	200
206	Bedsheet Set	Home	Spaces	1299	300
207	Laptop Stand	Electronics	AmazonBasics	899	180
208	Cushion Covers (Set)	Home	HomeCenter	599	400

7. Orders Table

Query

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  order_date DATE NOT NULL,  
  status VARCHAR(20) NOT NULL,  
  total_amount DECIMAL(12,2) NOT NULL,  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

Explanation

Stores order details.

Foreign Key:

Links each order to a customer.

Orders Sample Data

```
SELECT * FROM orders;
```

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498
1002	102	2024-08-03	Delivered	799
1003	103	2024-08-05	Shipped	7498
1004	101	2024-08-10	Delivered	3499
1005	104	2024-08-12	Cancelled	2999
1006	105	2024-08-15	Delivered	5898
1007	106	2024-08-18	Pending	1299
1008	103	2024-08-20	Delivered	899
1009	107	2024-08-25	Shipped	6098
1010	108	2024-08-28	Delivered	1598

8. Order Items Table

Query

```
CREATE TABLE order_items (  
    item_id INT PRIMARY KEY,  
    order_id INT NOT NULL,  
    product_id INT NOT NULL,  
    quantity INT NOT NULL CHECK (quantity > 0),  
    unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),  
    discount_pct DECIMAL(5,2) DEFAULT 0 CHECK (discount_pct BETWEEN 0 AND 100),  
    FOREIGN KEY (order_id) REFERENCES orders(order_id),  
    FOREIGN KEY (product_id) REFERENCES products(product_id)  
);
```

Explanation

Stores detailed product items of each order.

Each order can contain multiple products.

Order Items Sample Data

```
SELECT * FROM order_items;
```

item_id	order_id	product_id	quantity	unit_price	discount_pct
5001	1001	201	2	1499	0
5002	1001	207	1	899	10
5003	1002	202	1	799	0
5004	1003	203	1	2999	0
5005	1003	204	1	4599	5
5006	1004	205	1	3499	0
5007	1005	203	1	2999	0
5008	1006	201	1	1499	10
5009	1006	204	1	4599	5

item_id	order_id	product_id	quantity	unit_price	discount_pct
5010	1007	206	1	1299	0
5011	1008	207	1	899	0
5012	1009	205	1	3499	0
5013	1009	208	2	599	15
5014	1010	206	1	1299	0
5015	1010	208	1	599	0

9. Database Relationships

customers (1:N) orders

orders (1:N) order_items

products (1:N) order_items

Foreign Key Mapping:

customers.customer_id → orders.customer_id

orders.order_id → order_items.order_id

products.product_id → order_items.product_id

10. Indexes Used

Query

```
CREATE INDEX idx_customers_city ON customers(city);
CREATE INDEX idx_customers_state ON customers(state);
CREATE INDEX idx_products_category ON products(category);
CREATE INDEX idx_orders_date ON orders(order_date);
CREATE INDEX idx_orders_status ON orders(status);
```

Explanation

Indexes improve query speed when filtering large datasets.

Example:

- Searching customers by city
- Filtering products by category

- Filtering orders by date
-

11. Record Count Verification

Query

```
SELECT COUNT(*) FROM customers;  
SELECT COUNT(*) FROM products;  
SELECT COUNT(*) FROM orders;  
SELECT COUNT(*) FROM order_items;
```

Expected Output

Table	Total Records
Customers	8
Products	8
Orders	10
Order Items	15

Conclusion

The ShopEase E-Commerce Sales Analysis Database provides a strong foundation for learning SQL concepts.

Using this database, different analytical operations can be performed such as:

- Customer analysis
- Revenue analysis
- Product analysis
- Order tracking
- Inventory management
- Business reporting

This dataset is used in further assignment sections (A to E) for practical SQL query implementation.